课程笔记：构建 OpenHands 的经验与教训

Bowen-mantou（整理）

CMU 11-768: AI Agents（2026 年秋季）

主讲：Graham Neubig

2026 年 10 月 23 日

视频作者/频道：常青藤名校课（B 站）
发布日期：2026-09-09
视频时长：69:20
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=28
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 从研究到产品：这一讲为什么不同 **2**

1.1 一堂关于“真正被用户使用”的 agent 课 2

1.2 OpenHands 是什么：两条工作流 2

1.3 与 Claude Code、Codex 的关键差异 2

1.4 本章小结 3

2 历史：从代码补全到 agentic coding **3**

2.1 十年代码生成，两年 agent 3

2.2 Devin 时刻与开源复现 3

2.3 什么时候 agent 才真正可用 3

2.4 本章小结 4

3 软件 agent SDK：harness 的设计与教训 **4**

3.1 第二代 harness：好默认值，但全部可拆 4

3.2 自定义工具：action、observation 与 executor 5

3.3 隔离与安全：沙箱、确认策略与安全分析器 5

3.4 上下文管理：condensation（后来被改叫 compaction） 5

3.5 研究选型建议 6

3.6 本章小结 6

4 单 agent 还是多 agent **6**

4.1 2024 年的 specialized roles 风潮 6

4.2 单 agent 哲学：Don’t Sleep on Single Agent Systems 6

4.3 单 agent 的工程基础：通用工具、skills 与 MCP 7

4.4 单 agent 能有多通用：OpenHands-Versa 7

4.5 多 agent 仍然值得做的三个理由 7

4.6 本章小结 8

5 Agent 跑在哪、怎么交互 **8**

5.1 本地还是云：一个并不显然的答案 8

5.2 四种交互界面：IDE、GUI、CLI 与集成 8

5.3 人机协作的实证研究：更快，但更无聊、更难验证 9

5.4 本章小结 9

6 自动化与持续改进：把 agent 变成软件生产线 **9**

6.1 Automation 的形态：事件 → 有界任务 → 可检查的结果 9

6.2 两个真实用例：issue 分诊与 PR 评审 10

6.3 从历史运行中学习：Skill Refiner 10

6.4 下一步：从轨迹中挖掘工作流，以及严格评测 10

6.5 本章小结 11

7 总结与延伸 **11**
7.1 讲者的收尾 11
7.2 全讲结构压缩 11
7.3 开放问题与实践建议 12
7.4 拓展阅读 12

1 从研究到产品：这一讲为什么不同

1.1 一堂关于“真正被用户使用”的 agent 课

这是 Graham Neubig 在本课程中的最后一讲（后面是 Daniel Fried 的课与客座讲座）。开场他就说明，这一讲会和前面不太一样：一部分讲技术，但更多是讲他自己的经验——如何把一个 agent 产品真正做成有人用的东西，以及如何把“让用户真正用起来”的过程中遇到的痛点转化为研究课题。

这个定位很值得留意。前面十几讲大多在讲方法：怎么定义 agent、怎么设计 harness、怎么做评测、怎么做 RL 训练。而这一讲回答的是另一个问题：**当你把一个 agent 系统交给真实用户、放进真实工作流之后，哪些工程细节和研究问题才会暴露出来？**

1.2 OpenHands 是什么：两条工作流

OpenHands 是较早的开源软件开发 agent 之一，从 OpenDevin 改名而来。课上先演示了它的前端，核心是两条不同的工作流：

- **Human-in-the-loop**：在聊天窗口里输入你今天想做的任务，agent 在一个可交互的工作区里完成它；
- **Automation（自动化）**：把重复性工作配置成自动化任务，比如代码评审（code review）、仓库监控、Slack 频道监控等，让 agent 在后台持续执行。

演示里，Neubig 让 agent 去自己的开源仓库里找“今天需要处理的 open PR”，agent 会创建一段对话、访问仓库、读取 PR 信息并给出待办列表。他还展示了自动化面板：一个“每日工作流准备”的 automation 会把今天要做的事读出来；一个“软件工厂”式的 automation 集合里包含 PR 评审、issue 分诊（triage）、Slack 监控、依赖失败自动修复等。这两条工作流对应两种完全不同的产品形态，后面会分别展开。

1.3 与 Claude Code、Codex 的关键差异

Neubig 点出了 OpenHands 与大家熟悉的 Claude Code、Codex 的核心区别：后两者主要绑定在自己的语言模型生态里（分别偏向 Claude 和 GPT 模型），而 OpenHands 从设计上就尽量做到**模型无关（model-agnostic）**：它支持很多不同模型，可以临时切换模型再切回来，也可以自定义 MCP server、skills、plugins、apps。

另一个差异在“怎么用它”。OpenHands 是 Python 写的、面向“直接写代码调用它”的场景；OpenCode、Pi 等开源 agent 多用 TypeScript，且很多工具主要围绕 CLI 交互（很多 benchmark 就是通过命令行参数驱动 agent 跑起来的）。用 CLI 调用很方便，跨语言也能用，但一旦你想深入改造、实现新模块，命令行参数和配置文件的表达能力就会成为限制。因此选型时可以先问自己：你更想“当命令行工具用”，还是“当 Python 库改”？

Pi 的哲学：裸核 + 扩展

Neubig 顺带对比了 Pi 的设计思路：Pi 刻意只给一个非常精简的内核，然后靠扩展把功能加上去。OpenHands 的 SDK 走了相反的路：先给一套开箱即用的好默认值，但允许你把任何一部分拆掉替换。两种路线没有绝对优劣，反映的是“最小内核”与“默认体验”的优先级差异。

1.4 本章小结

- 这一讲的主题是“把 agent 做成产品”的经验：真实用户、真实工作流会暴露研究论文里看不到的问题。
- OpenHands 支持两条工作流：交互式的 human-in-the-loop，以及后台自动化的 automation；它的定位是模型无关、可用 Python 直接编程的 agent 平台。
- 与 Claude Code、Codex、OpenCode、Pi 等工具的差异，不在“谁更聪明”，而在模型生态绑定、语言与接口形态、可改造程度上。

2 历史：从代码补全到 agentic coding

2.1 十年代码生成，两年 agent

Neubig 从 2016 年就开始做代码生成，但直到 2023 年前后，主流做法仍然不是 agentic 的：主要是代码补全、生成单个函数。真正的 agentic code generation 大约从 2024 年才开始，而短短两年里，他自己已经不再手写代码了。

2023 年的一件大事是 SWE-bench 的发布：它把真实 GitHub issue、仓库快照、patch 和仓库测试组合在一起，成为一个仓库级的评测基准。它的影响力极大，但早期结果很差——大约 5% 的水平，几乎没人能拿到像样的分数。这个“看起来能做、实际做不出”的落差，正是 agent 研究要解决的问题。

2.2 Devin 时刻与开源复现

真正让局面改变的，是 2024 年 3 月 Devin 的演示。Devin 被称为“第一个 AI 软件工程师”：你输入任务，它会自己规划、开始工作。这段演示视频在很短时间内获得上百万播放。Neubig 当时已经做了几年 web agent 与 coding agent，也有八年代码生成的积累；他看完的第一反应是“我知道这件事是可能的”，但 Devin 把它包装得非常有吸引力。

问题在于 Devin 不开源。Neubig 一直是开源的支持者，于是萌生了“做一个类似但开源的东西”的想法。就在 Devin 演示的第二天，Qwen 团队的 Junyang Lin 与 Binyuan Hui 发出号召：一起复现 Devin。于是这个只有 README 的仓库迅速拿到一万 star，成为当时最受关注的项目之一。

Neubig 当天晚上用 Claude 的聊天窗口，照着一张 Devin 的截图，做出了第一版界面；大约三天后，就出现了第一个真正能跑起来、能完成任务的 agent 例子（OpenDevin，后来改名 OpenHands）。与此同时，SWE-bench 的作者们其实早就在做 coding agent，后来发布了 SWE-agent；只是发布节奏被 Devin 抢了一点先机。5 月，OpenDevin 发布 CodeAct 1.0，把“写代码执行”作为核心动作空间。整个领域在几个月内快速成形。

2.3 什么时候 agent 才真正可用

但必须诚实地说：在很长一段时间里，这些 agent 并不真的可用。Neubig 提到，Devin 团队自己也承认这一点——他们甚至在公交站打广告说“Devin, it’s actually good now”，这本身就说明之前不够好。从 2024 年到 2024 年 11 月左右，agent 的整体表现都很勉强。

转折点在 2024 年年底：Claude Sonnet 3.5/3.7 这一代模型让 agent 开始“真的能用”；2025 年则是 coding agent 全面变好的一年，到年底，不只是 Anthropic，OpenAI 等大公司也都有了很强

的 coding 模型。到了 2026 年，Neubig 的判断是：模型还在变强，但这一年的主战场是**部署与界面**——更长的运行时间、并行任务、更好的可视化、可重复的工作流、凭证管理与验证。

这段历史的用处不在于怀旧，而在于解释后面要讲的设计：很多看似“产品细节”的东西（比如上下文压缩、skills、automation、可验证性），都是过去一年半里从真实使用中被逼出来的经验，后来才被固化进框架。

2.4 本章小结

- 代码生成从 2016 年就开始了，但 agentic 形态是 2023—2024 年才出现的；SWE-bench（2023.10）用真实 issue 与仓库测试定义了“仓库级”问题，早期成绩只有约 5%。
- 2024 年 3 月 Devin 演示引爆关注，次日 Qwen 团队号召复现，OpenDevin（后为 OpenHands）在几天内跑通；SWE-agent（4 月）与 CodeAct 1.0（5 月）奠定了早期动作空间。
- 2024 年底之前 agent 普遍不好用；2024 年底到 2025 年模型变强后 agent 才真正可用；2026 年的关键在部署与界面。

3 软件 agent SDK：harness 的设计与教训

3.1 第二代 harness：好默认值，但全部可拆

OpenHands 的 harness 已经发展到第二代。Neubig 很坦白：第一代 harness 很糟糕，难用、难扩展。大约八个月前，他们干脆把原有实现全部推倒重来，围绕“可组合、可替换”重新设计。

这个 SDK 的设计哲学是：给一套真的很好用的默认值，开箱即用；但允许你把它们拆掉，替换成自己的实现。它覆盖 models、tools、state、workspace 等基础件，也允许自定义 model choice、新工具、instructions 和部署方式。GUI、CLI 和各种集成（integrations）都建立在这个 SDK 之上。

下面这段代码就是课上展示的最小配置思路（按幻灯片内容整理，省略了无关细节）：

```
llm = LLM(
    model="openai/gpt-5.5",
    api_key=os.getenv("LLM_API_KEY"),
)
agent = Agent(
    llm=llm,
    tools=[
        Tool(name=TerminalTool.name),
        Tool(name=FileEditorTool.name),
        Tool(name=TaskTrackerTool.name),
    ],
)
```

Listing 1: 用 OpenHands SDK 配置一个基础 agent（按幻灯片内容整理）

配置好 agent 之后，workflow 也很直接：指定 workspace（工作目录），创建一段 conversation，发消息，然后运行。这里 Neubig 特别提醒：**workspace 是灵活的**——你可以在本地目录、容器、远端沙箱里跑，这取决于你怎么做隔离。而 conversation 的本质是 tasks、actions、observations 三者的累积，再加上一个 action-observation 循环。

3.2 自定义工具：action、observation 与 executor

SDK 允许你实现自定义工具。做法是把“动作”和“观测”都定义成类型：例如实现一个 grep 工具，就定义 `GrepAction` 与 `GrepObservation`，再提供一个 executor——它接收一个 action，执行后返回一个 observation。这相当于给“工具”定义了一个通用接口：动作与观测是数据，executor 是把数据变成环境行为的函数。

```
class GrepAction(Action):
    pattern: str = Field(description="Pattern␣to␣search␣for")
    path: str = Field(default=".")

class GrepObservation(Observation):
    matches: list[str] = Field(default_factory=list)
```

Listing 2: 自定义工具：action 与 observation 的类型定义（按幻灯片内容整理）

这段设计看起来简单，但它把“工具”从一个黑盒函数变成了可记录、可回放、可比较的数据结构——这正是后面做评测、做训练、做自动化时需要的东西。

3.3 隔离与安全：沙箱、确认策略与安全分析器

Agent 会执行任意命令，所以隔离是绕不开的。课上讲了两种沙箱用法：一种是先准备好 sandbox，再把 agent 跑在里面；另一种是使用 Python 的 `with` 语句，让 with 块内执行的所有命令都进入沙箱。Neubig 说两种都不错，他没有明确结论，但“直接把 agent 跑在沙箱里”的生态工具更多一些。

除了沙箱，还有一层安全机制是**确认策略（confirmation policy）**：决定哪些动作可以自主执行，哪些必须停下来等人确认；以及**安全分析器（security analyzer）**：在每一步分析这个动作是否安全到可以自主执行。前者是流程控制，后者是风险判断，两者结合才能在“自主性”和“可控性”之间取平衡。

SDK 的完整能力还包括 secrets 管理、LLM metrics、LLM routing、planning agent、observability 等。Neubig 的提醒是：这个 SDK 非常全，全到课上只能讲很小一部分；如果你需要长 rollout、内建可观测性等能力，它很合适。

3.4 上下文管理：condensation（后来被改叫 compaction）

长会话必然要压缩上下文。Neubig 说他们很早就实现了这件事，最初叫 **condensation**（压缩/凝缩），后来 Anthropic 把同一件事叫作 **compaction**，于是业界大多改用了 compaction 这个名字，而 OpenHands 内部仍叫 condensation。

关于压缩，有几个值得记住的工程经验：

- 最直接的实现是“旧事件 → 摘要”，并把**最初的指令**保留在近处（`keep_first`），避免关键约束被摘要掉；
- 他们试过把单个动作掩掉（mask out individual actions）的做法，但效果不如摘要，现在业界普遍采用摘要式压缩；
- 何时压缩？可以每 n 轮压缩一次，也可以按 token 数压缩；一个常用阈值是把模型最大上下文窗口乘以约 3/4，超过就触发压缩。

3.5 研究选型建议

针对“做研究该用哪个 harness”，Neubig 给了很实用的建议：如果你用 Python 做 coding agent 研究，可以考虑功能完整的 OpenHands，或者极简的 mini-swe-agent——后者只有 bash 工具、代码很少，方便你直接改。你要权衡的是：需要长 rollout、内建可观测性等“能力”，还是需要“能快速 hack 的简单底座”。如果你用 TypeScript，或者只通过 CLI 驱动，其他 agent 可能更顺手。

他还分享了两个常见问题的答案。为什么用 Python？因为项目最初是机器学习背景的人做的，而且 Python 在“读 agent 写的代码”时更快、更直观。代价是类型安全（需要用严格类型检查和每次提交跑类型检查来兜底）和速度，但速度通常不是瓶颈——你等的是模型或 agent 运行的程序，而不是 agent 框架本身。

模型会不会“只对 OpenHands 过拟合”？

有同学问：模型是不是过拟合了 OpenHands 的结构？Neubig 的回答很坦然：确实有很多模型在 OpenHands 这类流行 harness 上训练过；他们也曾仔细研究 Claude 使用的工具集并尽量对齐（OpenHands 的工具与 Claude Code 很接近，因此和 Claude 模型配合很好）。但现在模型对新 harness 的适应能力已经不错，OpenAI 的模型对各种 harness 都能用；Gemini 曾经因为 harness 适配问题分数偏低，后来通过 multi-harness training 改善。总体结论是：主流模型现在大多开箱就能用。

3.6 本章小结

- 第二代 OpenHands SDK 的设计原则是“好默认值 + 全部可拆换”；它把 models、tools、state、workspace、conversation 做成可组合的积木，十行左右代码就能搭出一个 agent。
- 自定义工具 = action + observation + executor；conversation 是 tasks/actions/observations 的累积，workspace 决定隔离方式。
- 工程化的安全与上下文管理是刚需：沙箱、确认策略、安全分析器、condensation（compaction）；压缩时要保留关键指令，阈值常用最大上下文窗口的约 3/4。

4 单 agent 还是多 agent

4.1 2024 年的 specialized roles 风潮

OpenHands 起步的时候，多智能体正流行。一个典型做法是给 agent 分配固定角色：Manager 负责计划、Reproducer 负责复现 bug、Fault localizer 负责定位、Editor 负责改代码、Verifier 负责验证修复。Neubig 提到这来自一篇多智能体工作（CodeR；他现场先口误说成 MetaGPT，随即更正），当时围绕 PR 的解决流程出现了大约 15 种变体。

4.2 单 agent 哲学：Don’t Sleep on Single Agent Systems

面对这股风潮，Neubig 在 2024 年年底写了一篇博客《Don’t Sleep on Single Agent Systems》，核心论证是：语言模型正在变强，而固定工作流里的角色分工，本质上是把“先做这个、再做那个”的流程写死；如果模型的指令遵循能力足够好，那么用一段提示词描述流程，模型自己就能完成，不一定需要复杂的多智能体编排。

所以从第一天起，他就拒绝在多智能体系统上投入太多精力。事后看，这个判断大概率是对的：至少对 coding 来说，现在已经很少有人实现那种固定的多智能体工作流了。他也留了例外：客服等场景可能需要严格固定的流程，因为一旦 agent 自由发挥就可能给出错误信息、带来法律责任；但在 coding 这种需要灵活性的任务上，单 agent 的收益通常更大。

4.3 单 agent 的工程基础：通用工具、skills 与 MCP

单 agent 能成立，前提是工具足够通用。OpenHands 给 agent 的主要能力只有三件：编辑文件、运行命令、浏览网页（视觉检查与文档查阅）。他们不额外堆很多专用工具，但支持 MCP（Model Context Protocol）把外部服务接进来。

但只有通用工具还不够：如果所有情形的操作规范都写进单一系统提示，提示词会膨胀到难以维护（“推 GitHub 时这么做、用 Docker 时那么做……”）。OpenHands 的解法是 **skills**：把特定场景的提示词（有时还包括脚本）打包成技能，在需要时拉进上下文。有趣的是，他们很早就有这个机制，当时叫 microagents，后来 Anthropic 实现了同样的东西并起名 skills——Neubig 承认，这个名字确实比 microagent 好得多。

两者还有一个小差别：Anthropic 的做法是让 agent 自己选择要查哪个 skill，再读取对应文件；OpenHands 则用显式的关键词触发。无论哪种，目标都是一致的：**在正确的时间，把正确的上下文拉进来**。

这个思路也给出了多智能体的替代方案：多智能体是给每个角色固定提示词和固定工具；skills + MCP 则是在需要时动态拉入提示词与工具。单个 agent 因此也能覆盖“角色分工”带来的能力。

4.4 单 agent 能有多通用：OpenHands-Versa

为了验证“通用工具 + 多模态浏览”的路线，OpenHands 团队（包括这门课的助教 Aditya）做了一篇论文《Coding Agents with Multimodal Browsing Are Generalist Problem Solvers》。思路很简单：给 agent 代码能力，再给它多模态浏览（看图像、看网页）的能力，它就能解很多不同类型的任务，而不只是写代码。

论文的核心结论是：这套通用组合不仅可行，而且在多个原本由专用 agent 主导的任务上超过了当时的最好结果。这成为 OpenHands 一以贯之的哲学主线：与其写一堆专用角色，不如把通用能力做强。

4.5 多 agent 仍然值得做的三个理由

不过，Neubig 并没有把多智能体一棍子打死。随着任务越来越有野心，多智能体在某些场景下越来越有用，他给出三个理由：

1. **效率**：如果任务彼此独立（例如十个不同的 PR），可以并行分发；
2. **独立评审**：让一个 agent 检查另一个 agent 的工作。这里有一个很有价值的经验观察：如果让 agent 评审自己的工作，它会宽容得多，因为它拥有一条条决策的“理由”；而一个看不到这些理由、上下文被切断的独立 agent，评审会严厉得多——这通常正是检查所需要的；
3. **成本控制**：把任务派给子 agent，它完成后只报告重要信息、丢掉过程中的上下文；也可以用更便宜的模型做实现，把更聪明的模型留给编排。

多 agent 不是“白拿”的收益

Neubig 特别强调，多智能体系统的上下文管理非常难。他们有一篇论文研究异步软件工程 agent：一个 manager 把任务分派给多个 engineer，在 PaperBench、Commit0 这类很长的实现任务上运行。要让它真正工作，需要工程师 agent 反馈“恰到好处”的上下文、避免彼此冲突或重复实现同一个东西，manager 还要把各人的产出正确集成回代码仓库——这些都很难。最后虽然做成了，但过程相当艰难。与其说多 agent 是捷径，不如说它把难点从“单 agent 的能力”转移到了“协调与集成”。

课上还回答了同学的问题：多智能体怎么追踪 reasoning trace、怎么理解到底发生了什么？Neubig 的坦诚回答是：通常做得并不好；他自己的做法是直接问 manager，让 manager 去翻看 trace，因为它比自己快。多 agent 在 deep research 这类“把任务分发到大量文档源”的场景里更容易赢，但在需要精细集成的 coding 场景里，固定流程往往不值得。

4.6 本章小结

- 2024 年流行“固定角色 + 固定工作流”的多智能体；Neubig 主张单 agent：模型指令遵循变强后，一段提示词就能承载流程，coding 领域尤其如此。
- 单 agent 的工程基础是通用工具（编辑/运行/浏览）+ skills（按需拉入场景提示词）+ MCP（按需接入外部工具），目标都是“在正确时间拉入正确上下文”。
- OpenHands-Versa 论文说明通用能力可以跨任务取胜；多 agent 的价值主要在并行、独立评审、成本控制，但协调与集成成本很高，不是白拿的收益。

5 Agent 跑在哪、怎么交互

5.1 本地还是云：一个并不显然的答案

这是课上“实践部分”的第一个问题：agent 到底跑在你自己的电脑上，还是跑在云工作区里？

Devin 最初是云产品（SaaS）；最早的 OpenDevin 则跑在本地，但被隔离在 Docker 容器里——主要原因很现实：当时他们不信任 agent 直接在自己的机器上跑。Neubig 自己也曾认为本地模式不可能被接受：如果 agent 被 prompt injection，偷走你的数据或上传 API key 怎么办？

结果恰恰相反：人们非常喜欢在本地跑 agent。对人机协作型的日常工作（修 bug、实现功能），本地模式已经变成主流；而对非 coding 工作或后台工作（比如 code review），云工作区更流行。Claude Code 一类产品则在尝试混合形态：让 agent 跑在云上、却操作你本地的电脑。

本地执行的安全代价

本地模式好用，但它把风险留给了用户：如果 agent 被 prompt injection 或权限过大，它可能泄露本地数据、凭证或修改不该改的文件。这也是为什么 OpenHands 强调沙箱、凭证限制与确认策略——安全不能靠“agent 应该乖”，而要靠运行时机制。

5.2 四种交互界面：IDE、GUI、CLI 与集成

课上把当前主流的 agent 交互方式归为四类，并现场统计了同学们的使用情况：

表 1: 四类 agent 交互界面的特点（按课堂讲解整理）

界面	优势	局限或注意点
IDE	熟悉：文件、diff、测试、调试器、聊天与任务管理、评审都在一个地方	如果你已经不怎么读代码，IDE 某种程度上是“遗留界面”
GUI	为 agent 原生设计的桌面应用，界面可以围绕任务而非代码组织	使用习惯仍需培养，生态较新
CLI	到处都能用：服务器、家用机器、远端机器	交互形态偏文本，复杂可视化与评审体验有限
集成	在已有工作流（Slack、GitHub、code review）里直接使用，不用切换界面	依赖平台能力，定制空间较小

课堂上 CLI 是使用最多的（约六成），IDE 次之（约一半略少），GUI 很少。Neubig 认为 CLI 的优势是“哪里都能用”，而 IDE 的问题在于：当 agent 承担了大部分编码工作、你不再逐行看代码时，IDE 的很多功能并非为你设计。GUI 的想象空间在于重新设计“以 agent 为主、代码为辅”的工作界面。集成式（如 Copilot code review）则可能越来越普遍——最好的界面是你已经待着的界面。

5.3 人机协作的实证研究：更快，但更无聊、更难验证

OpenHands 团队做过一项人机研究，比较普通 Copilot 用户与使用 OpenHands agent 的用户在 Python 任务上的表现。结果是“天壤之别”：agent 用户大约用一半时间完成了两倍的任务，正确率也显著更高（图中约 0.6 对 0.25，时间约 12 分钟对 25 分钟）。

但负面反馈同样重要：受试者说“看着 agent 干活很无聊”，以及“很难确认它到底做得好不好”。这两个吐槽恰好指出了人机交互设计的两个核心问题：

- 既然人会无聊，产品就应该支持多任务——让一个人同时推进多条 agent 工作流；
- 既然验证困难，产品就必须帮助人验证输出——这不是“agent 能力”问题，而是界面与流程设计问题。

5.4 本章小结

- 本地 vs 云不是非此即彼：人机协作的日常 coding 偏本地，后台/非 coding 工作偏云；本地模式必须以沙箱与权限机制控制风险。
- 四类界面各有取舍：IDE 熟悉但可能“遗留”，GUI 面向 agent 重新设计，CLI 无处不在，集成式把 agent 嵌入已有工作流；课堂使用率 CLI 最高、IDE 次之。
- 人机研究显示 agent 显著提升效率（约一半时间、两倍任务、更高正确率），但“无聊”和“难验证”是两个必须由产品设计解决的问题。

6 自动化与持续改进：把 agent 变成软件生产线

6.1 Automation 的形态：事件 → 有界任务 → 可检查的结果

Neubig 说这是他们现在投入很多的方向：面向软件开发的自动化。一个 automation run 的结构可以概括为：一个触发事件（issue、PR 或定时器）进来，收集上下文，在隔离的工作区里运行，

产出一个可检查的 artifact，最后由人或策略来检查。

具体到配置，一个 automation 需要：触发方式与指令、仓库与权限、输入事件到运行的映射，以及对输出质量的检查。他们在 OpenHands 里维护了一套“软件工厂”式的 automation 集合：GitHub PR 评审、issue 分诊、依赖版本升级失败自动修复、Slack 消息触发等。

6.2 两个真实用例：issue 分诊与 PR 评审

Issue triage。 作为一个大型开源项目，OpenHands 会收到大量 PR——其中很多是为了丰富简历而用 AI 快速生成的（“vibe coded”）。这未必是坏事：别人花 token 帮你实现功能；但麻烦在于很多 PR 做得不对，因为 issue 本身描述不清。于是 issue triage bot 做两件事：一是判断 issue 是否足够清楚，如果不够清楚，就列出需要澄清的问题；二是如果 issue 可做，就给出“完成条件/验收标准”（acceptance criteria），让人和 agent 都清楚要做到什么程度。

PR review。 他们有一套非常细的代码评审规则：比如这个 PR 是否在本仓库的范围内、需要检查哪些点、在什么情况下可以批准、什么情况下必须请求修改。这些规则进入 code review bot，由它在每个 PR 上执行。

为什么要花力气做这两件事？因为实现层面可以交给社区贡献者，而“判断该不该做”和“做得对不对”是维护者的瓶颈。更重要的是：这些流程每天重复，天然适合被自动化，也天然适合被改进。

6.3 从历史运行中学习：Skill Refiner

一旦 automation 每天运行，你就有了一个持续产生的数据集：每次 triage 或 review 都会产生一条轨迹，并且事后能拿到“延迟反馈”——比如 bot 说“LGTM（看起来没问题）”，但维护者随后说“这很差，需要修改”。于是问题就变成了一个机器学习问题：如何从这些带反馈的历史轨迹里，自动改进下一次的 prompt / skill？

这就是他们最近发表的 Skill Refiner 的工作。流程大致是：

1. 把轨迹按结果分成成功与失败，并附带证据（例如“维护者否定了 PR”就是失败证据）；
2. 用 embedding 对成功轨迹、失败轨迹分别聚类，找出反复出现的问题模式；
3. 从每个簇里生成对 skill（指令）的修改提案；
4. 用证据门控过滤：只保留那些有支持证据、看起来真能改善下一次运行的提案；
5. 把正、负提案合并，生成修订后的 skill。

这比一些 baseline（课上提到 GEPA 与 T2S 等方法）更便宜也更好。

6.4 下一步：从轨迹中挖掘工作流，以及严格评测

Neubig 认为这个方向还有两块很大的空间：

- **从大量轨迹中挖掘 workflow**：现在的 automation 大多是人凭经验、声明式地写出来的；如果能聚类大量用户的使用轨迹，把常见用例自动变成 automation，再让它进入“自动改进”的闭环，系统就能持续变好；

- **改 prompt 之外，还可以训练模型**：修改 prompt 或 harness 是一种改进，修改模型参数是另一种改进；后者需要更严格的评测。为 issue triage、code review 这类“欠定义（under-specified）”任务设计严格评测很难，但价值很大——如果你能把评测做好，就等于拿到了持续改进这些系统的钥匙。

6.5 本章小结

- Automation 把开发流程变成“事件 → 有界任务 → 可检查产物”；OpenHands 用它做 PR 评审、issue 分诊、依赖修复、Slack 触发等。
- 重复运行的 automation 会积累带延迟反馈的轨迹，把“改进 agent”变成一个可以从数据中学习的问题；Skill Refiner 用聚类、提案与证据门控自动改进 skill，实验结果优于初始 skill 与 baseline。
- 未来的两个方向：从轨迹中挖掘工作流，以及为欠定义任务建立严格评测（这也是训练模型的前提）。

7 总结与延伸

7.1 讲者的收尾

这是 Neubig 在本课程的最后一次课。他最后提到，之后会是客座讲座与 Daniel 的课程，并祝大家秋假愉快。整讲真正想传递的，是一个从开源项目到公司、从研究到产品的完整经验：

- 一个好的 agent 产品，往往不是从“最强模型”出发，而是从**真实工作流**出发：哪些步骤重复、哪些判断是瓶颈、哪些结果需要被检查；
- 很多关键设计（上下文压缩、skills、sandbox、confirmation policy、automation）都是在真实使用中被迫形成的，后来才被抽象进框架；
- 开源与公司化并不矛盾：OpenHands 变成公司，是因为它足够流行、有人愿意为它全职投入；而目前最大的客户是希望在自己的云上安全运行 coding agent、需要模型选择与可定制性的企业；
- 研究选题可以从“用户痛点”中来：多智能体的协调成本、长任务的验证、欠定义任务的评测，都是真实系统里长出来的研究问题。

7.2 全讲结构压缩

把这一讲压缩成五句话：

1. **历史**：代码生成已有十年，但 agentic coding 只有两年；SWE-bench 定义了仓库级问题，Devin 引爆关注，开源复现与模型进步共同让 agent 在 2024 年底真正可用，2026 年的重点是部署与界面。
2. **harness**：好的 SDK 要给可用的默认值，同时允许全部拆换；工具 = action + observation + executor；会话 = tasks/actions/observations 的累积；沙箱、确认策略、安全分析器与上下文压缩（condensation/compaction）是工程刚需。

3. **单 agent 优先**：模型变强后，固定角色的多智能体工作流价值下降；通用工具 + skills + MCP 能在“正确时间拉入正确上下文”，单 agent 也能通用（OpenHands-Versa）。

4. **多 agent 有条件地用**：并行、独立评审、成本控制是三个正当理由；但协调与集成成本很高，独立评审甚至比自我评审更严厉，这是它真正的价值所在。

5. **自动化闭环**：automation 把开发流程变成可重复、可检查的生产线；历史轨迹 + 延迟反馈 = 可学习的改进信号（Skill Refiner）；最终目标是让 prompt、harness、模型与评测一起持续改进。

7.3 开放问题与实践建议

- **长任务的验证**：agent 做得越久，人越难确认它是否真的完成；如何设计“可检查的产物”与验证流程，是产品与研究的共同问题。
- **多智能体的协调**：如何让子 agent 的上下文“不多不少”、如何避免重复实现、如何把产出集成回仓库，仍是开放问题。
- **欠定义任务的评测**：issue triage、code review 这类任务的“好”很难量化；谁能把评测做好，谁就能把自动化做得更好。
- **从轨迹到工作流**：能否从真实使用轨迹中自动挖掘出 automation，而不是靠人凭经验手写？
- **成本与安全**：多 agent 不一定更省钱，本地执行不一定更安全；两者都需要显式的优化目标与运行时约束。

如果要把这一讲变成可执行的一步，建议是：选一个你每天重复的软件工程流程（比如 PR 评审或 issue 分诊），把它写成一条 automation（触发 → 上下文 → 隔离执行 → 可检查产物），然后开始积累轨迹与反馈——这就是把 agent 从“demo”变成“系统”的最短路径。

7.4 拓展阅读

- OpenHands（开源软件开发 agent）：`https://github.com/All-Hands-AI/OpenHands`
- OpenHands 文档：`https://docs.all-hands.dev/`
- mini-swe-agent（极简 harness）：`https://github.com/SWE-agent/mini-swe-agent`
- SWE-bench：`https://www.swebench.com/`